

Should You Even Train a Neural Network? A Forecasting-Based Early Rejection Strategy under Budget Constraints (draft)

Santiago Ramírez Castañeda

Universidad de La Sabana

Faculty of Engineering: Maestría en Analítica Aplicada

Félix Mohr

02 de septiembre de 2025

Should You Even Train a Neural Network? A Forecasting-Based Early Rejection Strategy under Budget Constraints (draft)

Abstract

Neural networks have demonstrated remarkable performance across diverse tasks, yet their design remains resource-intensive due to the vast search space of possible architectures and the high cost of full training. This thesis investigates whether early-training signals and lightweight evaluation strategies can be effectively used to predict whether neural network architectures will achieve competitive performance compared to strong non-neural models, thereby enabling a fast and resource-efficient neural architecture search (NAS) framework. To address this question, we propose Effort-Based Evolution for Neural Architecture Search (EBE-NAS), a method that introduces the concept of Effort Computation, where candidate architectures are allocated a constrained and uniform amount of training effort for evaluation. EBE-NAS integrates this principle within an evolutionary search strategy, balancing rapid exploration with selective refinement of promising architectures. The framework is evaluated against traditional machine learning baselines, fully trained oracle architectures, and strong non-neural models. Results demonstrate that EBE-NAS can reliably forecast and select architectures that approach oracle-level performance while remaining competitive with non-neural baselines, all at a fraction of the computational cost. These findings highlight the potential of early-training signals as a viable basis for efficient NAS.

Introduction

The pursuit of optimal machine learning models has long relied on iterative experimentation, expert intuition, and extensive computational resources. In deep learning, this process often translates into a costly trial-and-error cycle involving the manual tuning of architectures and hyperparameters (Salehin et al., 2024; Zeiler & Fergus, 2014). Although this approach can yield high-performing models, its inefficiency makes it impractical in resource-constrained or time-sensitive scenarios.

Automated Machine Learning (AutoML) frameworks emerged to alleviate these challenges by automating model selection and hyperparameter optimization. These systems aim to make high-performance modeling accessible without requiring deep expertise, balancing predictive accuracy and computational cost (Elsken et al., 2019; Mohr & Wever, 2023). Tools such as NaiveAutoML demonstrate that robust performance can be achieved through simplicity

and efficiency, avoiding black-box optimization or extensive search pipelines.

In the context of deep learning, Neural Architecture Search (NAS) extends the automation principle to the design of neural networks themselves. NAS algorithms—based on reinforcement learning, evolutionary computation, or Bayesian optimization—have demonstrated the ability to discover high-performing architectures across diverse tasks (Poyser & Breckon, 2024; Ren et al., 2022). However, despite their success, most NAS methods remain computationally expensive and difficult to reproduce, limiting their applicability in environments with restricted resources.

Despite their flexibility and expressive power, neural networks are not consistently selected over non-neural models in practical machine learning scenarios—particularly for tabular or structured data. Empirical benchmarks have repeatedly shown that gradient-boosted decision trees and related ensemble methods often outperform deep networks, offering stronger generalization, reduced sensitivity to preprocessing, and lower computational cost (Oyallon2022WhyTreeBased; Borisov2022DeepSurvey; WhenDoNNsOutperform2023; DeepLearningTabularIsNotAll2021). This persistent gap has motivated research into when, and under what conditions, neural architectures can become competitive alternatives to such baselines.

Recent work in neural architecture search (NAS) suggests that early-training dynamics can reveal strong indicators of eventual model performance. Lightweight evaluation strategies—including partial learning-curve forecasting, early-epoch predictors, and intrinsic architectural proxies—have shown that it is possible to approximate final accuracy using minimal training effort (Ru2021SpeedyPerformance; Wen2020NeuralPredictor; Luo2020SemiNAS; Chen2021BNNAS). Collectively, these advances point toward a new paradigm of resource-efficient NAS, where performance estimation and model selection occur long before full convergence.

Building on this idea, the present research investigates whether early-training signals and lightweight evaluation strategies can effectively predict whether neural network architectures will achieve competitive performance compared to strong non-neural baselines. The proposed *Effort-Based Evolution* (EBE) framework implements this hypothesis through a simplified

evolutionary process that evaluates neural candidates using minimal training while progressively discarding unpromising architectures. The approach aims to make NAS more practical, reproducible, and computationally accessible without sacrificing competitive performance.

Background

Automated Machine Learning and Model Selection

Introduce the motivation for automating model design and selection within the broader AutoML paradigm. Explain how AutoML systems streamline preprocessing, model search, and hyperparameter optimization to identify strong baselines efficiently. Highlight that such systems frequently converge on non-neural models—particularly gradient-boosted trees and random forests—because of their robustness and consistent performance across tabular datasets.

Performance of Neural Networks on Tabular Data

Discuss empirical findings showing that neural networks often underperform compared to tree-based ensembles in structured-data tasks. Summarize key studies demonstrating this gap, linking it to neural networks’ sensitivity to feature scaling, irregular distributions, and limited inductive bias for tabular structures. Establish that this performance disparity motivates a need to better understand *when* neural architectures are worth the training investment.

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ denote a dataset drawn from distribution $\mathcal{P}(X, Y)$. Given a hypothesis space $\mathcal{H}$ containing both neural ($\mathcal{H}_{\text{NN}}$) and non-neural ($\mathcal{H}_{\text{nonNN}}$) models, the model selection objective is

$$h^* = \arg \max_{h \in \mathcal{H}} \mathbb{E}_{(x,y) \sim \mathcal{P}} [M(h(x), y)], \quad (1)$$

where $M(\cdot)$ is a performance metric such as accuracy or negative loss. Empirical studies show that for tabular data, models $h \in \mathcal{H}_{\text{nonNN}}$ (e.g., gradient-boosted trees) often yield higher M than $h \in \mathcal{H}_{\text{NN}}$ under comparable optimization budgets.

Neural Architecture Search (NAS) and Efficiency Constraints

Introduce NAS as an attempt to automate neural design itself, reviewing major search strategies such as reinforcement learning, evolutionary algorithms, and Bayesian optimization. Emphasize their computational cost and limited accessibility, which make them less practical in low-resource or real-time settings. Position NAS as a potential bridge between neural flexibility and AutoML efficiency—if evaluation can be made lightweight enough.

Early-Training Signals and Lightweight Evaluation

Describe how recent research has focused on identifying predictive cues within the earliest stages of training. Outline methods such as early-epoch validation extrapolation, partial learning-curve forecasting, and zero-cost proxies that aim to predict model quality without exhaustive training. Cite evidence that these early indicators correlate strongly with final performance, supporting the feasibility of faster, more cost-aware model selection.

Given partial observations of a candidate’s validation performance $\{(t_j, a_j)\}_{j=1}^k$ up to time t_k , the goal is to estimate its asymptotic performance $\hat{a}_\infty$ via a lightweight predictor f_θ :

$$\hat{a}_\infty = f_\theta(\{(t_j, a_j)\}_{j=1}^k), \quad \text{with } \text{cost}(f_\theta) \ll \text{cost}(\text{full training}). \quad (2)$$

A reliable f_θ enables early termination of underperforming architectures, reducing the total search cost without sacrificing final accuracy.

Performance Forecasting and Decision-Making in NAS

Explain how forecasting methods estimate validation accuracy over time using fitted curves or probabilistic predictors. Clarify the role of these estimates in balancing exploration and exploitation within evolutionary or Bayesian NAS frameworks. Discuss the challenge of determining—*early and reliably*—whether a candidate network deserves continued training compared to existing non-neural baselines.

Motivation for the Epoch-Based Evolution (EBE) Framework

Synthesize the gap revealed by prior subsections: while non-neural models dominate current AutoML outcomes, existing NAS methods remain too costly to compete at scale. Argue that a decision-oriented, early-training approach could close this gap by evaluating neural candidates through minimal computation and rejecting unpromising ones early. Introduce EBE as a streamlined evolutionary framework designed to operationalize this idea—merging early-signal forecasting, selective continuation, and efficiency-driven architecture search.

The decision can be framed as a binary variable

$$\delta(h) = \begin{cases} 1, & \text{if } \mathbb{E}[M(h)] \geq M_{\text{baseline}}, \\ 0, & \text{otherwise,} \end{cases} \quad (3)$$

where M_{baseline} denotes the performance of the best non-neural model. The goal of EBE is to estimate $\delta(h)$ as early as possible using limited training evidence.

Problem Definition

Research Question

Can early-training signals and lightweight evaluation strategies be effectively used to predict whether neural network architectures will achieve competitive performance compared to strong non-neural models, enabling a fast and resource-efficient NAS framework?

Research Objectives

General Objective

To develop a fast and resource-efficient neural architecture search (NAS) framework capable of determining whether neural networks are likely to yield competitive performance compared to strong non-neural models, by leveraging early training information and lightweight evaluation strategies.

Specific Objectives

- Develop a lightweight forecasting approach that uses early training signals to estimate the probability of neural architectures achieving competitive accuracy relative to strong non-neural baselines.
- Integrate evolutionary search and probabilistic scoring into the NAS framework that progressively refines candidate architectures under strict time and data budgets.
- Validate the proposed framework empirically by benchmarking its ability to identify competitive neural architectures versus non neural models, assessing both predictive accuracy and resource efficiency.

Methodology

Overview of the NAS Framework

The proposed NAS toolkit is designed to identify top-performing neural network architectures with minimal computational resources. The workflow consists of candidate architecture generation, predictive evaluation using early-training signals, evolutionary search for optimization, and benchmarking against strong non-neural models.

The first methodological component focuses on developing a forecasting mechanism capable of predicting the long-term performance of neural network architectures from minimal training information. This step addresses the need for rapid evaluation of candidate models while avoiding the cost of full convergence.

Data Preparation. All datasets are obtained from the OpenML repository, following a simple standardized preprocessing pipeline. Categorical features are encoded through label encoding, while continuous features are normalized using standard scaling. The data are subsequently split into training, validation, and test sets. Target variables are preprocessed by collapsing rare or unseen classes into a common `__other__` bucket before applying label encoding, ensuring consistent handling across splits.

Search Space Configuration. The design of candidate architectures is guided by a broad but controlled search space, which specifies the range of architectural and training hyperparameters. This configuration balances diversity—to ensure exploration of a wide set of structural patterns—with practicality, by constraining values to ranges that are computationally feasible. Table 1 summarizes the standard configuration adopted throughout the experiments.

Table 1

Standard configuration of the search space for candidate architectures.

Hyperparameter	Configuration
Number of hidden layers	2 – 25
Neurons per layer	3 – 100
Activation functions	ReLU, LeakyReLU, Sigmoid, Tanh, ELU, GELU
Dropout rates	0.0, 0.1, 0.2, 0.3, 0.4, 0.5
Learning rate	Uniform in log scale, 10^{-4} – 10^{-1}
Batch size	Powers of two: 32, 64, 128, 256, 512, 1024
Optimizers	Adam, SGD, RMSprop, AdamW
Weight decay	0, 10^{-6} , 10^{-5} , 10^{-4} , 10^{-3} , 10^{-2}
Momentum	0.8, 0.9, 0.95, 0.99 (only for SGD)
Skip connections	True, False
Initializers	Xavier (uniform/normal), Kaiming (uniform/normal)
Learning rate schedulers	Step, Exponential, Cosine, None
Architecture shapes	Constant, Pyramid, Inverted Pyramid, Hourglass, Triangular, Irregular

The ranges and options in the search space were selected to balance expressiveness and efficiency: they are broad enough to cover diverse architectural patterns (e.g., deep and shallow networks, varying connectivity, different regularization strategies), while remaining bounded to avoid excessively large or impractical configurations that would undermine the time–efficiency goals of the framework.

Candidate Architecture Generation. The initial pool of neural networks is derived from the predefined search space that specifies the range of architectural and training hyperparameters. For each candidate, a random sampling procedure instantiates an architecture configuration, which is then used to construct a dynamic multilayer perceptron model. This procedure ensures diversity within the initial population, allowing the forecasting mechanism to be tested across a

broad spectrum of architectural profiles. The sampled architectures are subsequently paired with individual seeds to guarantee reproducibility.

Effort Representation. We define *effort* as the total wall-clock training time accumulated by a candidate model. Each mini-batch contributes its measured runtime, which is summed into a continuous timeline. This replaces simple epoch or batch counts: it naturally reflects differences in batch size, data subsampling, model complexity, and hardware throughput. Formally, for a candidate c with batch runtimes t_b , the effort is

$$\text{Effort}(c) = \sum_{b \in \mathcal{B}(c)} t_b$$

where $\mathcal{B}(c)$ denotes the sequence of training batches processed by c .

Early Training Protocol. Each candidate architecture is trained under a restricted instance budget using the following scheme. At every anchor, the model is exposed only to a subset of the training data, with this budget expanding progressively across generations. An epoch in this context corresponds to a complete pass over the currently allocated instances rather than the full dataset. After each epoch, preliminary metrics are recorded, including validation accuracy, training accuracy, and training loss, logged against the cumulative effort.

Forecasting from Early Training Signals

To reduce the cost of NAS, candidate performance is not only measured on the limited training budget allocated, but also extrapolated into the future through a forecasting module. The objective is to predict whether a neural candidate is likely to surpass strong non-neural baselines within the remaining time budget.

Given a candidate architecture, we record validation accuracy values $y = \{y_1, \dots, y_t\}$ observed at cumulative training efforts $x = \{x_1, \dots, x_t\}$, where effort is measured as total processed mini-batches. Based on these trajectories, the following steps are performed:

1. **Forecast horizon:** A candidate-specific time horizon T_f is defined as

$$T_f = t_{\text{now}} + N \cdot \max(t_{\text{passed}}, t_{\text{dataset}}),$$

where t_{now} is the cumulative elapsed time, t_{passed} is the equivalent time of all instances processed so far, and t_{dataset} is the time for one full pass of the dataset. The factor N (e.g., $N = 10$) ensures that forecasts project sufficiently beyond the observed window.

2. **Curve fitting:** Validation accuracy is extrapolated to T_f using a conservative regression model. In practice, rational and sigmoid fits are used:

$$f_{\text{rational}}(x) = \frac{ax}{b+x}, \quad f_{\text{sigmoid}}(x) = \frac{L}{1 + \exp(-k(x - x_0))}.$$

For robustness, forecasts are blended with the most recent observed value and penalized if the slope indicates stagnation.

3. **Confidence intervals:** Curve fitting additionally yields a mean forecast $\hat{y}_f$ with an associated confidence interval $[\hat{y}_{\text{low}}, \hat{y}_{\text{high}}]$, bounding the plausible range of future accuracy.
4. **Probability of improvement:** The probability that a candidate will surpass a given goal metric g is computed via a hybrid of Monte Carlo sampling and sigmoid-based scoring:

$$p = \alpha \cdot \Pr(\mathcal{N}(\hat{y}_f, \sigma^2) > g) + (1 - \alpha) \cdot \sigma\left(\frac{\hat{y}_f - g - \delta}{\tau}\right),$$

where α controls the Monte Carlo vs. analytic weighting, $\sigma(\cdot)$ is the logistic function, τ is a temperature parameter, and δ penalizes flat slopes or high forecast variance.

This probability p is then used as the candidate's score in the evolutionary selection step. By prioritizing individuals with both strong current performance and a high probability of surpassing the baseline, the method balances exploration with efficient early discarding of hopeless models.

Post-Evolution Fidelity Validation

After the evolutionary search concludes, the framework determines whether a neural network is worth further training relative to strong non-neural baselines. The reference threshold is the validation accuracy of the best baseline model (see Section ??), which serves as the minimum acceptable performance.

Decision Function. After each generation, the framework assesses whether continued neural exploration is justified. This is achieved through a Bayesian-inspired expected-utility criterion that jointly considers the likelihood of improvement, the magnitude of potential gain, and the remaining computational cost. The decision function produces a scalar utility estimate U defined conceptually as:

$$U = \mathbb{E}[\text{Benefit}] - \mathbb{E}[\text{Cost}],$$

where the expected benefit represents the probability-weighted advantage of a neural model surpassing the baseline performance, and the expected cost reflects the anticipated training effort still required.

Decision Function. Formally, the decision mechanism integrates three quantities:

- p : the estimated probability that the best current candidate will outperform the baseline.
- Δ : the expected validation-accuracy margin between the forecasted performance and the baseline.
- C : the normalized computational cost representing the remaining training budget.

The expected utility is defined as:

$$U = p \cdot \Delta - \lambda C,$$

where λ regulates the relative weight assigned to computational cost.

When $U > 0$, the framework interprets this as sufficient evidence that the expected improvement justifies continued investment. This first decision occurs *within* the evolutionary search loop: if no candidate achieves positive expected utility before the global time budget

expires, the process concludes that training neural networks is not worthwhile under the given constraints.

Triggering Extended Training. If the best candidate by forecast accuracy achieves $U > 0$, the evolutionary process halts and the model is selected for refinement. This phase employs a conventional early-stopping strategy constrained by the remaining global time budget. The model continues training until validation performance ceases to improve or the time budget is exhausted.

Early-Stopping Refinement Phase. During this phase, the selected candidate is optimized using early stopping with fixed patience and tolerance parameters. Training terminates automatically when any of the following conditions are satisfied:

1. Validation accuracy surpasses the baseline metric.
2. No measurable improvement is observed within the patience window.
3. The remaining global time budget is exhausted.

All progress, including validation performance, training evolution, and time usage, is recorded in an *extension log*. The final model corresponds to the checkpoint that achieved the highest validation accuracy within the allowed budget.

Final Decision. The framework therefore produces two sequential decisions:

1. An **internal decision** inside the EBE loop, determining whether a neural candidate is worth extended training ($U > 0$).
2. A **final validation-based decision** after extended training, verifying whether the selected candidate actually surpasses the non-neural baseline in validation accuracy.

If no candidate triggers the first decision, the process ends immediately. If extended training is triggered but the model ultimately fails to exceed the baseline, EBE concludes that neural networks are not competitive under the given budget. Test-set evaluations may optionally be performed for diagnostic purposes, but these are not part of the operational decision rule.

Validation and Benchmarking

Baseline Models and Metrics

To establish a consistent reference point, a non-neural benchmark was constructed using NaiveAutoML (Mohr & Wever, 2023), which evaluates a diverse set of classical models such as Gradient Boosting, Random Forest, and Logistic Regression. The hyperparameter optimization phase was intentionally skipped to preserve reproducibility and comparability, returning the best pipeline found up to that point. Neural models were explicitly omitted from this benchmark to isolate the non-neural performance ceiling. The validation accuracy of the best non-neural pipeline serves as the goal metric for EBE’s probabilistic decision layer.

In addition, two standard reference models were trained independently for each dataset: a HistGradientBoostingClassifier (HGB) and a Multilayer Perceptron (MLP). Their wall-clock training times and test accuracies were recorded to define computational constraints for subsequent neural search experiments. The effective time budget for EBE-NAS was determined by:

$$T_{\text{budget}}^{(\text{EBE})} = \max(T_{\text{HGB}} \cdot \beta_{\text{budget}}, T_{\text{MLP}}, 60),$$

where T_{HGB} denotes the training duration of the HistGradientBoosting baseline, β_{budget} is an adjustable multiplier controlling exploration depth (set to three in these experiments), T_{MLP} represents the training time of the neural baseline, and 60 seconds establishes a minimum allocation for datasets with small baseline costs.

Reproducibility and Experimental Setup

All experiments were conducted under deterministic random contexts. The utilities `init_global_seed()` and `make_repro_context()` enforce synchronized seeding across Python, NumPy, and PyTorch, while each DataLoader worker receives a derived seed to ensure reproducible sampling and batch ordering. This configuration guarantees that identical seeds yield equivalent evolutionary trajectories, identical ledger outputs, and consistent population dynamics across runs.

Data Preparation. Each dataset was retrieved from OpenML and split into training, validation, and test partitions with an 80/10/10 ratio using stratified sampling. Categorical features were label-encoded, numerical features standardized, and missing values imputed using the mean or mode depending on data type. All features and targets were subsequently converted to PyTorch tensors.

Population and Search Space. For every dataset, a search space $\mathcal{S}$ of feed-forward neural architectures was instantiated based on the input and output dimensionalities. The population size was fixed at $N = 25$, with candidates sampled uniformly from $\mathcal{S}$ (see Section Search Space ??) Training was executed on PyTorch with full determinism enabled.

Training Configuration. Each candidate began training with a limited instance budget corresponding to 10% of the available training data with the already described behavior in the increase. Validation and training metrics from these short runs were collected at batch level, serving as inputs for the forecasting and probabilistic scoring layers of EBE-NAS.

Evaluation Protocol. After the evolutionary phase concluded, the most promising candidate (if selected) was retrained until either (i) the validation accuracy surpassed the baseline metric or (ii) the time budget $T_{\text{budget}}^{(\text{EBE})}$ was exhausted. The final model was then evaluated on both validation and test sets. For each dataset, the validation and test accuracies, total time consumed, probability of surpassing the baseline, and expected utility at the decision phase, were recorded.

Outputs and Logging. Each experiment produced detailed outputs including (i) per-generation population dynamics, (ii) forecast-versus-fidelity comparisons, and (iii) comprehensive summaries integrating baseline metrics, time budgets, evolutionary statistics, and final decisions on whether the neural model was worth training under the given constraints for later analysis.

Results

Discussion

Referencias

- Elsken, T., Metzen, J. H., & Hutter, F. (2019). *Neural Architecture Search: A Survey* (tech. rep.). <http://jmlr.org/papers/v20/18-598.html>.
- Mohr, F., & Wever, M. (2023). Naive automated machine learning. *Machine Learning*, 112(4), 1131–1170. <https://doi.org/10.1007/s10994-022-06200-0>
- Poyser, M., & Breckon, T. P. (2024). Neural architecture search: A contemporary literature review for computer vision applications. *Pattern Recognition*, 147, 110052. <https://doi.org/10.1016/j.patcog.2023.110052>
- Ren, P., Xiao, Y., Chang, X., Huang, P.-y., Li, Z., Chen, X., & Wang, X. (2022). A Comprehensive Survey of Neural Architecture Search. *ACM Computing Surveys*, 54(4), 1–34. <https://doi.org/10.1145/3447582>
- Salehin, I., Islam, M. S., Saha, P., Noman, S., Tuni, A., Hasan, M. M., & Baten, M. A. (2024). AutoML: A systematic review on automated machine learning with neural architecture search. *Journal of Information and Intelligence*, 2(1), 52–81. <https://doi.org/10.1016/j.jiixd.2023.10.002>
- Zeiler, M. D., & Fergus, R. (2014). Visualizing and Understanding Convolutional Networks. In *Lecture notes in computer science (including subseries lecture notes in artificial intelligence and lecture notes in bioinformatics)* (pp. 818–833, Vol. 8689 LNCS). Springer Verlag. https://doi.org/10.1007/978-3-319-10590-1_{_}53